

TFG Reunion 29 abril

no constexpr
→ a estático

- # ① Contaminar header con smart pointers.
- el autocompletado se llena más
 - expone la implementación privada.
 - ~~preservar el espacio de compilación de lenguaje~~

② Sintaxis verbosa * vs. smart pointers sobre todo en el header file

③ ¿Cuándo usar smart pointers?
caso donde no estoy usando: ¿bien?

U32 registros [99];

U32 registros_activos [16]; ←

constexpr registros_indices [3][16] = {

{0, 1, 2, 3, ..., 15},

{0, ..., 7, 16, ..., 19, 15},

{20, ..., 34, 15},

uno de registros_indices
se copia a registros_activos

este es un array "incrustado" en
el struct
¿sería más óptimo si fuera
U32 * en lugar de copiar
16 * U32

④ ¿Por qué la lista de inicialización importa?
¿Por qué debe ser en orden?
def = def inicialización con punto d. a=1, c=2, b=3

⑤ Preguntar si macros en C++ está bien.
- como constantes
- como una forma más cómoda de evitar escribir código
↳ generar setters y getters que acompañen
receta (procedimiento) pero ~~distos~~ distintos ingredientes
(entradas)

⑥ Mi idea con el algoritmo para lot de instrucciones

- M instrucciones. Todas ocupan n bits
- extraer x bits de n.
- esos x bits identifican que instrucción llamar
- rellenar la tabla con eficiencia, pero una

misma instrucción puede repetirse

meto instrucción (llamada, ~~id~~ id_base, máscara, extracción)

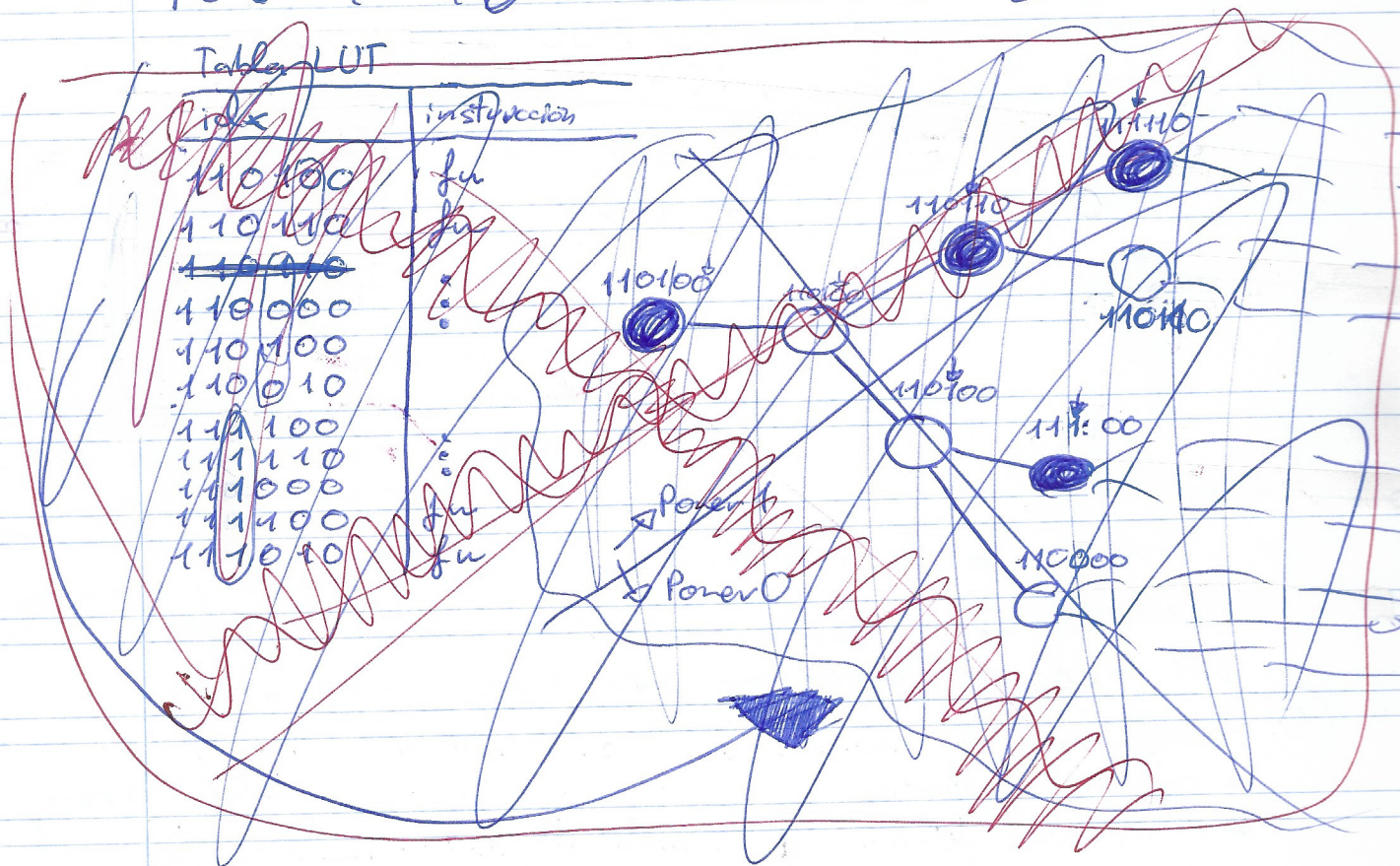
bits a ignorar

id_base | 01101000 00110111
 máscara | 00011010 00000000
 extraer | 01111011 00000000

esta máscara es común a todas las instrucciones y solo extrae los bits de interés

de forma que solo trabajamos

id_base | 1101 - 00
 máscara | 0011 - 10 → meto instrucción

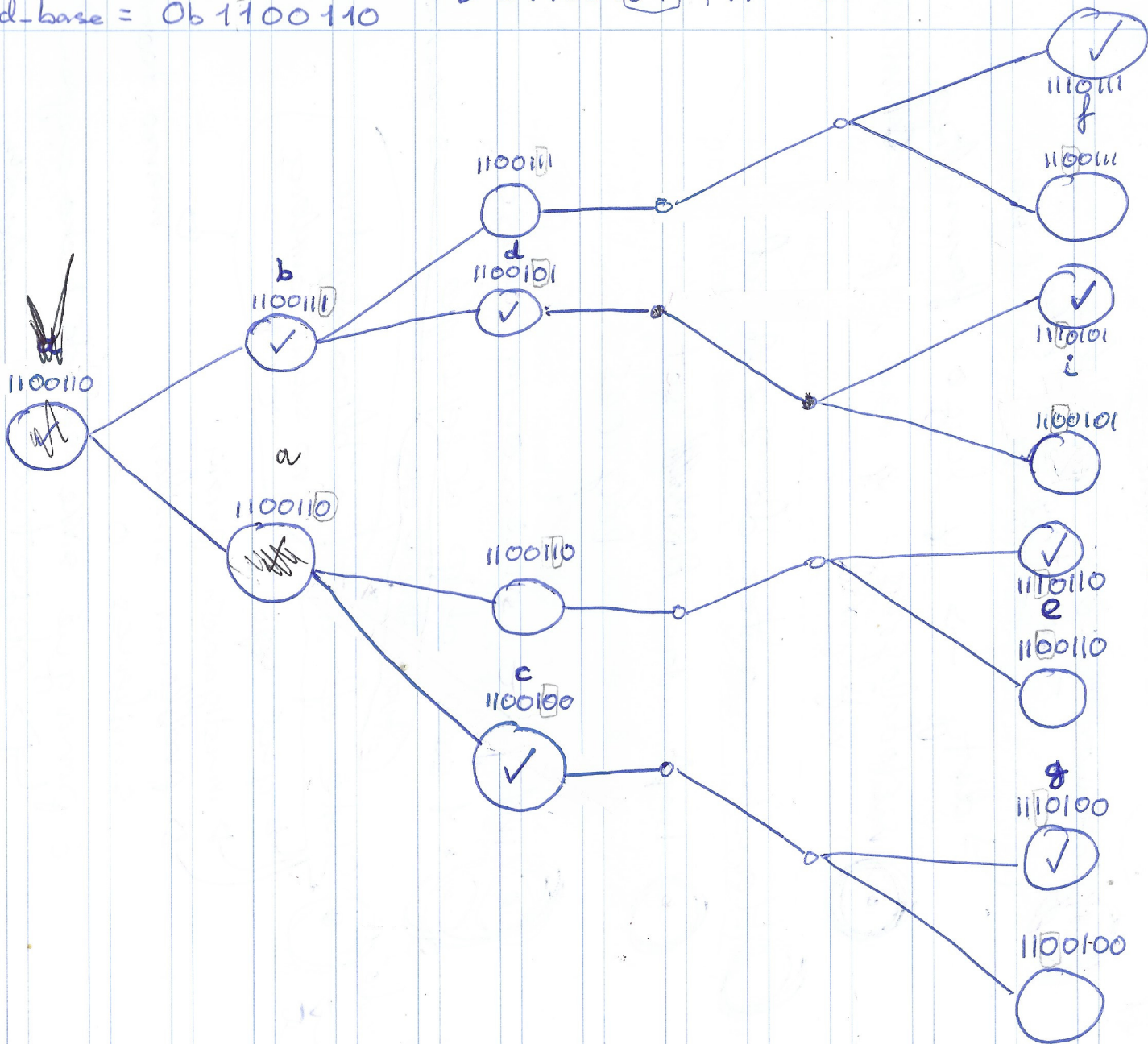


⑦ ¿Es mejor LUT o un switch?

depende del compilador
 y del switch en sí
 Mirar el ensamblador

el compilador cuando lo transforma? y cómo

$$\begin{array}{cccccc} a & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ b & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ c & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ d & 1 & 1 & 0 & 0 & 1 & 0 & 1 \end{array}$$



Compare and contrast
in various components
of James to
James to

⑧ Prepara per la documentar
accordan come hanno
contender
junter
blado

• planificar 30%

• iterativa cerrada producto

• margen grande

• decir qué funcionalidades
decidir

• ¿Cada cuánto verse?

• ¿Cada cuánto ^{hacen} documentación?

- al cabo de ^{hacen} ~~hacer~~

27 de julio 12:00

• Documentación como en blog
- don't take smart notes
- build your second brain

→ dirty → y luego formalizar

• metodologías: ~~que~~ ^{que} esas, procedimientos,
reflexión

• Hacer antes de 27 de jul

• Implementar ARM set
lógicas

• Interface memoria clase

• 1 solo chip de ejemplo

sinacrida

↑
"iniciando"

situar donde está

State of Art

público objetivo
no saben mucho
C++

→ qué hago

→ investigación propia

→ problema similar
similares o =

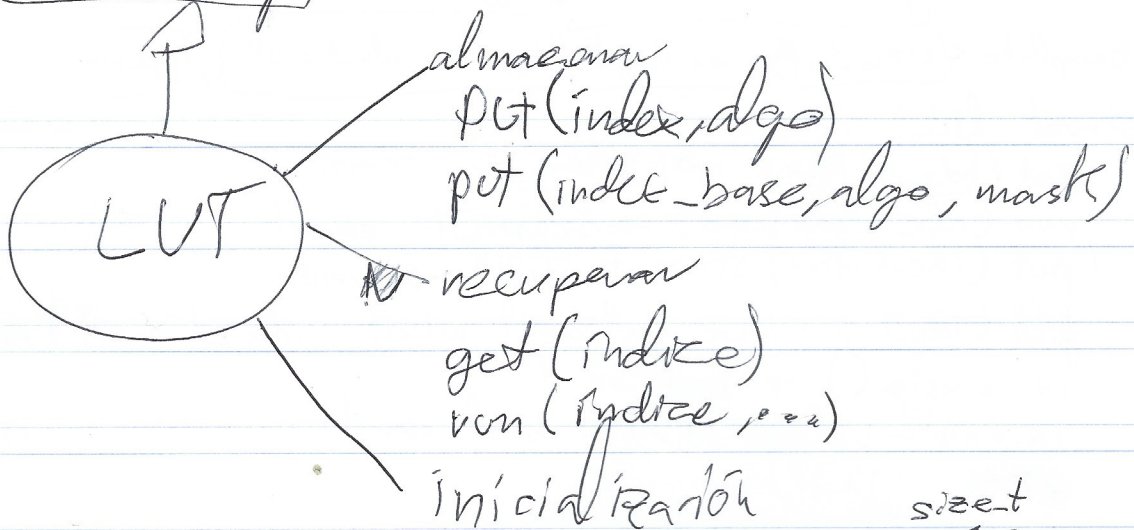
↓
marco técnico

¿para qué sirve si está hecho antes?

→ objetivo

comprobación

std::array < tamaño, tipo >



```

template <typename idx_t, typename store_t, int size_t n_max>
struct lut {
  idx_t n_max = static static_cast<idx_t>(n_max);
  std::array<n_max, store_t> storage;

```

~~virtual~~ ~~const~~ ~~get~~ ~~set~~ ~~run~~ ~~put~~ ~~is_empty~~ ~~count~~

struct lut

get(idx_t idx) → store_t const

run(idx_t idx, ...) → return_t const

put(idx_t idx, store_t what) → void

put(idx_t idx_base, store_t what, ~~int~~ idx_t mask) → void

is_empty() → bool const

is_empty(idx_t idx) → bool const

virtual ~~store_t~~ ~~run~~ ~~put~~ ~~is_empty~~ ~~count~~ ~~normalise~~ (idx_t idx) ^{const}

~~size_t~~ ~~count~~ count() ~~const~~ → size_t const ^{size_t}

count(store_t coincidence) → size_t const

② typename store_t = (void*)(u32))

template <① size_t ~~max_length~~ ③ typename idx_t = u8, ④ typename return_t = void>

struct lut {

get(① idx_t idx) → ③ store_t const

run(idx_t idx, ...) → ④ return_t const

put(idx_t idx, store_t what) → void

fill(idx_t idx_base, store_t what, idx_t mast) → size_t

is_empty() → bool const

is_empty(idx_t idx) → bool const

~~length~~ length() → size_t const

count_by_stones(store_t what) → size_t const

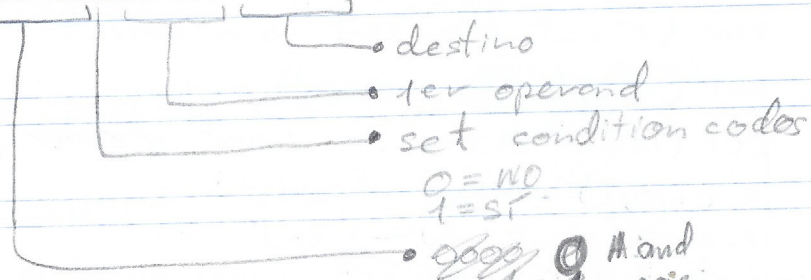
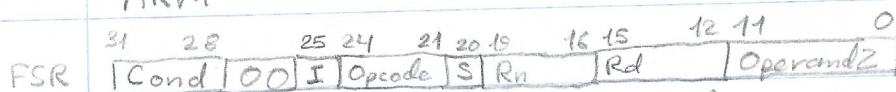
count_by_idx(idx_t idx) → size_t const

virtual normalize(idx_t idx) → size_t

erase(idx_t idx) → void

std::array

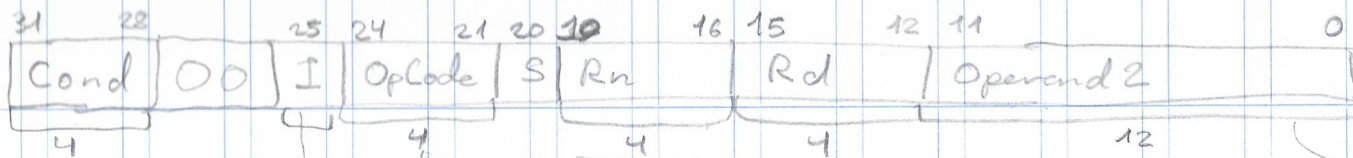
ARM



0 = NO
1 = ST

• 0000
• 0001
• 0010
• 0011
• 0100
• 0101
• 0110
• 0111
• 1000
• 1001
• 1010
• 1011
• 1100
• 1101
• 1110
• 1111

0x10547218



destino

1er operando

0 and
1 eor
2 sub
3 rsb

4 add
5 adc
6 sbc
7 rsc

8 tst
9 teq
A cmp
B cmn

12 ~~orr~~
13 ~~mov~~
14 ~~bit~~
15 ~~mvn~~

$i=0 \rightarrow$ registro



$i=1 \rightarrow$ inmediato

